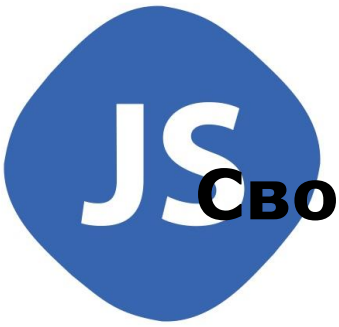


Свойства и методы массивов 1

- Введение
- Методы массива
- Перебор массива

Введение

Массивы предоставляют множество свойств и методов для работы с элементами.

> ***Примечание***. В свою очередь, элементом массива может быть что угодно (простой тип, сложный тип, функция и др.)

В предыдущих уроках мы работали со свойством **length** и методами добавления / удаления элементов массива:

- arr.**push**(...items) – добавляет элементы в конец
- arr.**unshift**(...items) – добавляет элементы в начало
- arr.**pop**() – извлекает элемент из конца
- arr.**shift**() – извлекает элемент из начала

В этом уроке рассмотрим другие часто используемые методы.

> ***Важно*.** Методы **добавления** / **удаления** элементов и методы **перебора** элементов массива чрезвычайно важны при разработке веб-приложений.

Но сначала несколько не рассматриваемых нами ранее особенностей свойства **length**.

length

length - свойство содержит общее число элементов массива.

Свойство **length** автоматически обновляется при изменении массива. Если быть точными, это не количество элементов массива, а **наибольший цифровой индекс плюс один**.

Например, единственный элемент, имеющий большой индекс, даёт большую длину.

example_1. Свойство length

```
<script>

    // инициируем пустой массив

    let authors = new Array();

    // присваиваем элементу массива значение

    authors[20] = "А.П. Чехов";

    alert(authors.length); // 21

</script>
```

Обратите внимание, назначать индексы элементам массива подобным образом - плохая практика.

Ещё один интересный факт о свойстве **length** – его можно **перезаписать**.

Если мы вручную увеличим его, ничего интересного не произойдёт. Зато, если мы уменьшим его, массив станет короче. Этот процесс необратим, как мы можем понять из примера.

example_2. Перезапись свойства length

```
<script>

    let arr = [1, 2, 3, 4, 5];

    console.log ("Исходный массив: ", arr);

    // укорачиваем массив до двух элементов
```

```
    arr.length = 2;
    console.log ("Измененный массив: ", arr); // [1, 2]
     // возвращаем исходную длину length
    arr.length = 5;
    console.log ("Восстановить массив нельзя: ", arr); // [1, 2,
    ...]
    // длину массива восстановили, данные нет
    console.log (arr.length);
</script>
```

Методы массива

isArray()

Массивы не являются отдельным типом данных языка JavaScript. Массивы являются видом объектов.

```
console.log(typeof {}); // object
console.log(typeof []); // object
```

Но массивы являются неотъемлемой частью любого приложения, поэтому существует специальный метод на проверку принадлежности переменной массиву.

Метод **isArray**() возвращает **true**, если объект является массивом и **false**, если он массивом не является.

Синтаксис

```
Array.isArray(obj);
```

Параметры

- **obj -** объект для проверки.

example_3. Проверка на принадлежность к массиву

```
<script>
    // инициируем массив из смешанных элементов
    let courses =[
        {id: 1, name: "JavaScript разработчик с нуля", price: 40000},
        [2, "Python-разработчик", "12 месяцев", 55000],
        [3, "Data Scientist", 48000],
        {id: 4, name: "Фронтенд-разработчик", price: 78000},
        {id: 5, name: "Тестирование веб-приложений на Python", price: 40000},
        {id: 6, name: "Разработчик C++", price: 126000},
        [4, "Фронтенд-разработчик на JavaScript", 65000],
        {id: 7, name: "Android-разработчик", price: 168000}
    ]
    // переберем элементы внешнего массива
    for (i=0; i < courses.length; i++) {
        if (Array.isArray(courses[i])) console.log(`Элемент ${i} - массив`);
    }
</script>
```

concat()

Метод **concat**() — **конкатенация** массивов, возвращает новый массив, состоящий из массива, на котором он был вызван, соединённого с другими массивами и/или значениями, переданными в **качестве аргументов**.



Синтаксис

```
arr.concat(arg1, arg2...);
```

Параметры

- **arg1**, **arg2** – массивы и/или значения, соединяемые в новый массив.

Метод **concat**() создаёт новый массив, состоящий из элементов массива, на котором он был вызван, за которыми по порядку следуют, для каждого аргумента, все его элементы (если аргумент является массивом), либо сам аргумент (если он массивом не является).

Метод **concat**() **не изменяет исходный массив** или любой из массивов, переданных в аргументах, а вместо этого возвращает поверхностную копию, содержащую копии тех элементов, что были объединены с исходными массивами.

example_4. Метод concat()

```
<script>
    // инициируем первый массив
    let rus = ["Толстой", "Достоевский", "Набоков", "Чехов"];
    console.log ("Русская литература: ", rus);
    // инициируем второй массив
    let  west = ["Шекспир", "Фолкнер", "Диккенс", "Флобер",
    "Остин"];
    console.log ("Западная литература: ", west);
    // складываем массивы
    let world = rus.concat(west, "Булгаков");
    console.log ("Мировое литературное наследие: ", world);
</script>
```

slice()

Метод **slice**() — создает новый массив из элементов исходного массива с индексами указанного диапазона. Говоря другими словами - **возвращает подмассив указанного массива**.

Синтаксис

```
arr.slice([begin[, end]]);
```

Параметры

- **begin** – необязательный. Индекс (счёт начинается с нуля), по которому начинать извлечение.
  - Если индекс отрицательный, **begin** указывает смещение от конца последовательности. Вызов slice(-2) извлечёт два последних элемента последовательности.
  - Если **begin** не определён, **slice**() начинает работать с индекса 0.
  - Если **begin** больше длины последовательности вернётся пустой массив.
- **end** – необязательный. Индекс (счёт начинается с нуля), по которому заканчивать извлечение. Метод **slice**() извлекает элементы с индексом меньше **end**.
  - Если индекс отрицательный, **end** указывает смещение от конца последовательности.
  - Если **end** опущен, **slice**() извлекает все элементы до конца последовательности (arr.**length**).

Описание

Метод **slice**() не изменяет исходный массив, а возвращает новую одноуровневую копию, содержащую копии элементов, вырезанных из исходного массива.

example_5. Метод slice

```
<script>

    // инициируем исходный массив

    var arr = new Array("php", "mysql", "laravel", "javaScript",
```

```
    ["css", "bootstrap"]);
    // создаем подмассивы из исходного массива
    let serv = arr.slice(0,3);
    let client = arr.slice(3,);
    // выводим полученные массивы
    console.log ("Изучаем серверные технологии: ", serv); // php,
    mysql, laravel
    console.log ("Изучаем клиентские технологии: ", client); //
    javaScript, [css, bootstrap]
</script>
```

splice()

Метод **splice**() изменяет содержимое массива, удаляя существующие элементы и/или добавляя новые. Метод используется для вставки и удаления элементов из массива.

Синтаксис

```
array.splice(start[, deleteCount[, item1[, item2[, ...]]]]);
```

Параметры

- **start** – индекс, по которому начинает изменять массив. Если больше длины массива, реальный индекс будет установлен на длину массива. Если отрицателен, указывает индекс элемента с конца.
- **deleteCount** – необязательный. Целое число, показывающее количество старых удаляемых из массива элементов.
    - Если **deleteCount** равен 0, элементы не удаляются. В этом случае вы должны указать как минимум один новый элемент.
    - Если **deleteCount** больше количества элементов, оставшихся в массиве, начиная с индекса **start**, то будут удалены все элементы до конца массива.

- **itemN** – необязательные параметры. Добавляемые к массиву элементы. Если вы не укажете никакого элемента, **splice**() просто удалит элементы из массива.

Возвращаемое значение

Массив, содержащий удалённые элементы. Если будет удалён только один элемент, вернётся массив из одного элемента. Если никакие элементы не будут удалены, вернётся пустой массив.

Описание

Если количество указанных вставляемых элементов будет отличным от количества удаляемых элементов, массив изменит длину после вызова.

В следующем примере удалим три элемента и возвратим массив из удалённых элементов.

example_6. Метод splice на примере простого массива

```
<script>
    // инициируем исходный массив
    var arr = new Array("php", "mysql", "laravel", "javaScript",
    "css", "bootstrap");
    // удаляем и возвращаем три элемента, начиная с индекса 0
    let trash = arr.splice(0,3);
    console.log ("Мы изучаем клиентские технологии: ", arr); //
    javaScript, css, bootstrap
    console.log ("А это мы выбросили на помойку: ", trash); //
    php, mysql, laravel
    console.log ("... это шутка была ;)");
</script>
```

В демонстрационном примере **example_7 splice**() удаляет два элемента и два элемента добавляет.

example_7. Метод splice на примере массива объектов

```
<script>
    // новый объект для добавления в массив
    let car = {
        id: 7,
        name: "Kia Sorento Prime",
        brand: "Kia",
        dvs_type: "Дизельный",
        image: "images/kia-sorento-prime.png"
    };
    // два элемента массива удаляем
    // два элемента добавляем
    let del = cars.splice(0, 2, {
        id: 6,
        name: "Chevrolet Tahoe",
        brand: "Chevrolet",
        dvs_type: "Бензиновый",
        image: "images/chevrolet-tahoe.png"
    }, car);
    // конечный массив
    console.log ("Конечный массив: ", cars);
    // массив удаленных элементов
    console.log ("Массив удаленных элементов: ", del);
</script>
```

Метод **splice**() также может **вставлять элементы без удаления**, для этого достаточно установить **deleteCount** в 0.



split()

Метод **split**() разбивает объект **String** на массив строк путём разделения строки указанной подстрокой. Метод возвращает **новый массив**.

Синтаксис

```
str.split([separator[, limit]]);
```

Параметры

- **separator** – необязательный параметр. Указывает символы, используемые в качестве разделителя внутри строки. Параметр **separator** может быть как строкой, так и регулярным выражением.
- **limit** – необязательный параметр. Целое число, определяющее ограничение на количество найденных подстрок. Метод **split**() всё равно разделяет строку на каждом сопоставлении с разделителем **separator**, но **обрезает возвращаемый массив** так, чтобы он содержал не более **limit** элементов.

example_8. Метод split

```
<script>
    // инициируем строку
    let strTags = " #PHP #JavaScript #CSS #MySQL";
    // разобьем строку в массив
    arr = strTags.split(" #");
    // отбросим первый элемент из массива
    arrTags = arr.slice(1, );
    // конечный массив выведем в консоль
    console.log (arrTags);
</script>
```

example_9. Метод split с управляющим символом \n

```
<script>
    // инициируем строку разделенную символом переноса строки: \n
    let albums = `2; Meddle; 30 октября 1971; EMI, Harvest,
    Capitol; Vinyl, Кассета, CD; Платиновый (USA),
    3; Obscured by Clouds; 3 июня 1972; EMI, Harvest, Capitol;
    LP, Кассета, CD; Золотой (USA), Серебряный (GBR),
    4; The Dark Side of the Moon; 17 марта 1973; Harvest,
    Capitol, EMI; LP, Кассета, CD, SACD; Платиновый (USA),
    Платиновый (GBR), Бриллиантовый (CAN)`;
    // разобьем строку в массив
    arr = albums.split("\n");
    // вывод элементов массива
    console.log ("Элемент 0:", arr[0]);
    console.log ("Элемент 1:", arr[1]);
    console.log ("Элемент 2:", arr[2]);
</script>
```

join()

Метод **join**() — объединяет все элементы массива в строку.

Синтаксис

```
arr.join([separator]);
```

Параметры

- **separator** – необязательный. Определяет строку, **разделяющую элементы** массива. Если параметр не задан, элементы массива разделяются запятой ','. Если разделитель - пустая строка, элементы массива ничем не разделяются в возвращаемой строке.



Описание

Преобразует все элементы массива в строки и объединяет их в одну большую строку. Элемент массива с типом **undefined** или **null** преобразуется в **пустую** строку.

example_10. Метод join

```
<script>
    // инициируем массив
    let tags = new Array("PHP", "JavaScript", "CSS", "MySQL");
    // преобразуем массив к строке
    // 1.
    console.log ("-> ", tags.join(", ")); // PHP, JavaScript, CSS,
    MySQL
    // 2.
    console.log ("-> ", tags.join(" ")); // PHP JavaScript CSS MySQL
    // 3.
    console.log ("-> ", tags.join("\n")); //
    PHP\nJavaScript\nCSS\nMySQL
    // 4.
    let line = "#" + tags.join(" #");
    console.log ("-> ", line) // #PHP #JavaScript #CSS #MySQL
</script>
```

delete()

Есть еще один метод для удаления элемента из массива.

Так как массивы – это особый тип объектов, то к ним можно применить метод **delete**().

example_11. Метод delete

```
<script>
    // инициируем исходный массив
    var arr = new Array("php", "матрешка", "javaScript",
    "балалайка", "css", "революция");
    // удаляем из массива лишнее
    delete arr[1];
    delete arr[3];
    delete arr[5];
    // удалили не сам элемент, удалили его значение
    console.log ("А осадочек остался... ", arr);
    // остался осадочек и размер массива
    console.log ("Размер массива: ", arr.length);
</script>
```

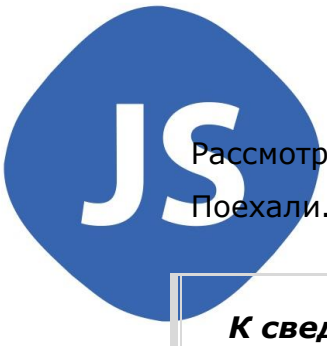
Элементы удалены, но количество элементов массива не изменилось. Метод **delete** хорош для объектов, но для массивов мы обычно хотим, чтобы оставшиеся элементы сдвинулись и заняли освободившееся место. Мы ждём, что массив станет короче.

По этой причине оптимальнее использовать специальные методы.

Перебор массива

Ранее мы рассмотрели классические циклы и их разновидность – специальные циклы **for...of** и **for...in**. Но массивы обладают еще рядом методов, предназначенных специально для обхода его элементов. Эти методы чрезвычайно полезны и очень часто будут вами использоваться.

Все рассматриваемые нами методы обхода элементов обладают одной особенностью – одинаковый набор входных параметров. Но есть и отличия.



Рассмотрим подробно практику применения в этом и следующих уроках. Поехали.

> ***К сведению*. Функции** и их разновидность **callback-функции** будем подробно изучать в следующей теме. Пока следует относиться к функции как к фрагменту кода, оформленному особым образом.

Метод **forEach** позволяет запускать функцию для каждого элемента массива.

Синтаксис

```
arr.forEach(function(elem, index, array) {
    // ...
});
```

Параметры

- **function(elem, index, array)** – callback-функция, которая будет вызвана для каждого элемента массива. Она принимает от одного до трёх аргументов:
  - **elem** – текущий обрабатываемый элемент в массиве.
  - **index** – необязательный. Индекс текущего обрабатываемого элемента в массиве.
  - **array –** необязательный. Массив, по которому осуществляется проход.

Благодаря возможности написания **короткой версии callback-функции** синтаксис может быть записан следующим образом:

```
arr.forEach((elem, index, array) => {
    // ...
});
```

А в силу того, что параметры **index** и **array** необязательные, синтаксис может быть таким:

```
arr.forEach((elem, index) => {
    // ...
});
```

Или даже таким:

```
arr.forEach( elem => {
    // ...
});
```

Возвращаемое значение – **undefined**.

Описание

Метод **forEach**() выполняет функцию callback один раз для каждого элемента, находящегося в массиве в порядке возрастания. Она не будет вызвана для удалённых или пропущенных элементов массива. Однако, она будет вызвана для элементов, которые присутствуют в массиве и имеют значение **undefined**.

В демонстрационном примере **example_12** обратите внимание на доступность переменных **item**, **index**, **teams** в теле цикла.

example_12. Перебор элементов простого массива

```
<script>
    // инициируем двумерный массив
    let teams = new Array(
        [1, "Aerosmith", "США", "1970", "хард-рок, блюз-рок, рок-н-ролл, глэм-метал"],
        [2, "Pink Floyd", "Великобритания", "1965", "психоделический-рок, арт-рок, эйсид-рок, прогрессивный-рок"],
        [3, "The Beatles", "Великобритания", "1960", "рок, рок-н-ролл, поп, хард-рок"],
        [4, "AC/DC", "Австралия", "1973", "хард-рок, блюз-рок,
```

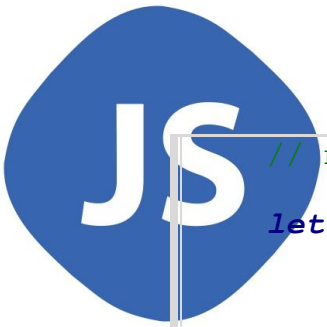
```
            рок-н-ролл"],
            [5, "Scorpions", "ФРГ", "1965", "хард-рок, глэм-метал,
            хеви-метал"],
            [6, "Ленинград", "Россия", "1997", "рок, ска, фолк, панк,
            панк-рок"]
    );
    let line = "";
    // выполняем перебор элементов массива teams
    teams.forEach ((item, index, teams) => {
        // проверяем доступность переменных item, index, teams
        console.log(teams[index][1]);
        line += `<li>${item[1]} (${item[2]})</li>`;
    })
    // исходный массив
    console.log(teams);
    // вывод в браузер сформированного списка
    document.write ("<ul>", line, "</ul>");
</script>
```

Результат функции, если функция что-то возвращает, будет **отброшен** и **проигнорирован**. Для возврата результата в функциях предусмотрено ключевое слово **return**. Ключевое слово **return** завершает работу функции и возвращает значение, которое является результатом вызова данной функции.

В демонстрационном примере **example_13** обратите внимание на использование синонима **myArr** для названия массива **teams** в теле цикла.

example_13. Перебор массива объектов

```
<script>
```



```
    // исходный массив объектов
    let teams = new Array(
        {id:1, /*...*/ },
        {id:3, /*...*/ },
        {id:4, /*...*/ },
        {id:5, /*...*/ },
        {id:6, /*...*/ }
    );
    newArray = Array();
    // результат метода forEach игнорируется
    // в теле цикла массив teams доступен через переменную myArr
    newArray = teams.forEach ((item, index, myArr) => {
        // вычеркнем Британию из списка демократических стран
        if (item.country == "Великобритания") {
            // заменим элемент исходного массива
            myArr[index] = "There is no democracy in this
            country";
            // что-то вернем (не важно что)
            return item.name;
        }
    })
    // заменили элементы массива текстовым значением
    console.log(teams);
    // метод не возвращает результат
    console.log(newArray);  // undefined
</script>
```





В теле функции **forEach** можно выполнять операции любой сложности.

В демонстрационном примере **example_14** обратите внимание на использование **вложенного цикла forEach** в теле внешнего.

example_14. Вложенный цикл forEach

```
<script>

    // исходный массив объектов
    let teams = new Array(
        {id:1, /*...*/ },
        {id:3, /*...*/ },
        {id:4, /*...*/ },
        {id:5, /*...*/ },
        {id:6, /*...*/ }
    );
    let line = "";
    // переберем элементы массива
    // не забывайте, каждый элемент массива - объект
    teams.forEach ( item => {
        line += `<li>${item.name}</li>`;
        subline = "";
        // массив стилей группы выведем подсписком второго уровня
        item.style.forEach (el => {
            subline += `<li>${el}</li>`;
        })
        subline = `<ul>${subline}</ul>`;
        line += subline;
    })
```

```
    // исходный массив
    console.log(teams);
    // сформированный двухуровневый список
    document.write ("<ol>", line, "</ol>");
</script>
```

Итого:

- Метод **forEach**() не возвращает результат.
- Элементы исходного массива легко **модифицировать**.

Если вам важен возвращаемый результат перебора элементов массива, можно использовать методы, рассматриваемые в следующем уроке.

В **самостоятельных работах** к этому уроку:

- Объединение массивов цепочкой вызовов.
- Проверка на принадлежность к массиву элементов вложенного массива.
- Обработка данных связанных массивов.
- Работа со сложными объектами данных (структура объекта Регионального чемпионата).

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).



К каждой теме курса:

- **Практические** работы